

But there is something wrong with the output, right? The shortest path from 0 to 1 is shown as passing through vertex 2, and the distance is shown to be just 2. This suggests that while calculating the shortest path and its length, the code in lines 3 and 4 ignores the edge weights, leading to an incorrect result. Upon closer inspection of the graph *Wgraph_17*, it can be seen that the shortest path from 0 to 1 is actually 0–2–3–1, and the length of this path is 66. Now, we need to find the shortest path considering the edge weights. To do that, replace lines 3 to 5 of the program ‘*wgraph17.sagews*’ with the following three lines of code (line numbers have been included for clarity in the explanation).

```
3. shortest_path = G.shortest_paths(0, by_weight=True,
algorithm="Dijkstra_Boost")
4. shortest_path_len = G.shortest_path_lengths(0, by_
weight=True, algorithm="Dijkstra_Boost")
5. print(shortest_path, '\n', shortest_path_len, '\n',
shortest_path_len[1])
```

Now, let us go through the code line by line to understand how it works. Line 3 calculates the shortest paths from vertex 0 to all other vertices in the graph *G*. The first parameter (0 in this case) specifies the source vertex from which the paths are calculated. The parameter ‘*by_weight=True*’ ensures that the shortest paths are determined based on the sum of the edge weights (i.e., the actual distance or cost) rather than just the number of edges. The parameter ‘*algorithm="Dijkstra_Boost"*’ specifies that Dijkstra’s algorithm should be used to compute the shortest paths (more on Dijkstra’s algorithm soon). The result, stored in *shortest_path*, is a dictionary where each key is a vertex, and the corresponding value is a list representing the shortest path from vertex 0 to that vertex.

Line 4 calculates the shortest path lengths (i.e., the total weight or distance) from vertex 0 to all other vertices in the graph *G*. The parameters and their meanings are the same as those used in Line 3. The result, stored in *shortest_path_len*, is a dictionary where each key is a vertex, and the corresponding value is the total weight (length) of the shortest path from vertex 0 to that vertex.

Line 5 prints the results of the shortest path calculations. It prints the dictionary containing the shortest paths from vertex 0 to all other vertices, the dictionary containing the shortest path lengths (distances) from vertex 0 to all other vertices, and the shortest path length (distance) specifically from vertex 0 to vertex 1.

Upon executing the modified program, the following additional output will be displayed on the screen along with *Wgraph_17*.

```
{0: [0], 1: [0, 2, 3, 1], 2: [0, 2], 3: [0, 2, 3]}
{0: 0, 1: 66, 2: 11, 3: 33}
66
```

Now, let us analyse this output. The first line shows the *shortest_path* dictionary ({0: [0], 1: [0, 2, 3, 1], 2: [0, 2], 3: [0, 2, 3]}), which details the shortest path from vertex 0 to each of the other vertices. For example, for vertex 0, the path is [0], meaning the shortest path from 0 to itself is trivially just 0. For vertex 1, the path is [0, 2, 3, 1], indicating that the shortest path from 0 to 1 passes through vertices 2 and 3. The second line shows the *shortest_path_len* dictionary ({0: 0, 1: 66, 2: 11, 3: 33}), which represents the corresponding shortest path lengths (i.e., the total weight of the paths) from vertex 0 to each of the other vertices. For example, the distance from vertex 0 to itself is 0, as expected, since no travel is involved. The distance to vertex 1 is 66, which corresponds to the sum of the edge weights along the path [0, 2, 3, 1]. This is calculated as 11 (from 0 to 2) + 22 (from 2 to 3) + 33 (from 3 to 1) = 66. Finally, the third line shows the specific shortest path length from vertex 0 to vertex 1, which, as mentioned earlier, is 66.

Now, a word about Dijkstra’s algorithm, developed by one of the greatest computer scientists of all time, Edsger W. Dijkstra (pronounced ‘Dykstra’). In his honour, the most influential paper in distributed computing is awarded the annual Dijkstra Prize. Dijkstra developed this algorithm in 1956 to find the shortest path in a graph when all edge weights are positive (and yes, edge weights can also be negative). The algorithm identifies the shortest path from a given source vertex to every other vertex in the graph. A variant of Dijkstra’s algorithm, known as uniform cost search, is widely used in artificial intelligence.

In the modified ‘*wgraph17.sagews*’ program discussed earlier, the parameter *algorithm* in lines 3 and 4 offers several options beyond *Dijkstra_Boost*, including *BFS*, *Dijkstra_NetworkX*, and *Bellman-Ford_Boost*. It is important to note that *Dijkstra_Boost* (the Boost Graph Library implementation in C++) and *Dijkstra_NetworkX* (the NetworkX implementation in Python) are two different implementations of Dijkstra’s algorithm. But how do the other algorithms differ from Dijkstra’s? They vary based on the types of graphs they can handle, their execution speed, and other factors. For example, the Bellman-Ford algorithm can handle graphs with negative edge weights, as long as there is no cycle with negative weight. The study of distances in graphs is a significant field in computer science with countless applications. Although a detailed discussion is beyond the scope of this series, I strongly encourage you to explore this area further on your own.

Next, we will dive into searching in graphs. Whether an AI-based application is navigating a maze, playing chess, or deciding what to do next in a conversation, it is all about searching for the best option among countless possibilities. In AI, graph search is key to finding optimal or most efficient solutions. For instance, in path-finding algorithms like